

数据结构作业 2 答案

1、

```
#define MAXSIZE 100

#include<stdio.h>

typedef struct{
    int data[MAXSIZE];
    int length;
}SeqList;

void initSeqList(SeqList*list){
    list->length=0;
    int i=0;
    int n=0;
    while(scanf("%d",&i)&&i!=-1&&n<MAXSIZE){
        list->data[n++]=i;
        list->length++;
    }
}

int insert(SeqList*list,int x){
    if(list->length==MAXSIZE) return -1;
    int n=0;
    while(list->data[n]<x){ n++;
    }
    int i=list->length;
    while(i>n){
        list->data[i--]=list->data[i-1];
    }
    list->data[i]=x;
    list->length++;
    return 0;
}
```

```

}

int main()
{
SeqList list;
initSeqlist(&list);

int x;
scanf("%d",&x);
insert(&list,x);
return 0;
}

```

2、

```

#include<stdio.h>
#include<stdlib.h>
typedef struct Node{
int data;
struct Node*next;
}Node;
Node* creatL(Node*l){
Node*trail=NULL;
Node*head=l;
int i=0;
while(scanf("%d",&i)&&i!=-1){
Node*newNode=(Node*)malloc(sizeof(Node));
newNode->next=NULL;
newNode->data=i;
if(head==NULL) head=trail=newNode;
else{
trail->next=newNode;
trail=newNode;
}
}
}

```

```

return head;

}

Node* deleteL(Node*l,int mink,int maxk){
Node dummy;
dummy.next=l;
Node*p=&dummy;
while(!p->next){
<#code#>
if(p->next->data>mink&& p->next->data<maxk){
Node*q=p->next;
p->next=q->next;
free(q);
}
else{p=p->next;
}
}
return dummy.next;
}

int main(){
Node*l=creatL();
creatL(l);
int mink,maxk;
scanf("%d %d",&mink,&maxk);
l=delete(l);
return 0;
}

```

2、改进版：

```

#include <stdio.h>

#include <stdlib.h>

typedef struct Node {
int data;

```

```

struct Node* next;

} Node;

// 创建链表：输入 -1 结束

Node* creatL() {
    Node *head = NULL, *trail = NULL;

    int val;

    while (scanf("%d", &val) && val != -1) {
        Node* newNode = (Node*)malloc(sizeof(Node));
        newNode->data = val;
        newNode->next = NULL;
        if (head == NULL) {
            head = trail = newNode;
        } else {
            trail->next = newNode;
            trail = newNode;
        }
    }

    return head;
}

// 核心算法：时间复杂度 O(n)

void deleteRange(Node** headRef, int mink, int maxk) {
    if (headRef == NULL || *headRef == NULL) return;

    Node dummy;
    dummy.next = *headRef;

    Node* p = &dummy;
    while (p->next) {
        if (p->next->data > mink && p->next->data < maxk) {
            // 找到符合条件的，删除并释放空间

            Node* q = p->next;
            p->next = q->next;
            free(q);
        }
    }
}

```

```

} else if (p->next->data >= maxk) {
// 因为是有序链表，一旦大于等于 maxk，后面肯定都不符合条件
break;
} else {
// 只有没发生删除时，指针才向后移动
p = p->next;
}
}
// 关键一步：将 dummy.next 指回给 main 里的头指针
*headRef = dummy.next;
}
// 打印链表
void printL(Node* l) {
while (l) {
printf("%d -> ", l->data);
l = l->next;
}
printf("NULL\n");
}
int main() {
printf("请输入有序链表元素（-1 结束）：\n");
Node* L = creatL();
int mink, maxk;
printf("请输入 mink 和 maxk：\n");
scanf("%d %d", &mink, &maxk);
deleteRange(&L, mink, maxk); // 传地址
printf("删除后的链表：\n");
printL(L);
return 0;
}

```

2、版本二：

```
#include <stdio.h>

#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

// 创建链表逻辑相同

Node* creatL() {
    Node *head = NULL, *trail = NULL;
    int val;
    while (scanf("%d", &val) && val != -1) {
        Node* newNode = (Node*)malloc(sizeof(Node));
        newNode->data = val;
        newNode->next = NULL;
        if (head == NULL) head = trail = newNode;
        else {
            trail->next = newNode;
            trail = newNode;
        }
    }
    return head;
}

// 核心算法：通过返回 Node* 来更新头指针
Node* deleteRange(Node* l, int mink, int maxk) {
    if (!l) return NULL;
    Node dummy;
    dummy.next = l;
    Node* p = &dummy;
    while (p->next) {
        if (p->next->data > mink && p->next->data < maxk) {
            Node* q = p->next;
```

p->next = q->next; // 这里修改的是 dummy.next 或者中间节点的 next

free(q);

} else if (p->next->data >= maxk) {

break; // 递增有序，提前结束

} else {

p = p->next;

}

}

// 把修改后的实际头指针返回

return dummy.next;

}

void printL(Node* l) {

while (l) {

printf("%d -> ", l->data);

l = l->next;

}

printf("NULL\n");

}

int main() {

printf("请输入有序链表 (-1 结束): \n");

Node* L = creatL();

int mink, maxk;

printf("请输入 mink 和 maxk: \n");

scanf("%d %d", &mink, &maxk);

// 重点：必须接收返回值并赋值给 L

L = deleteRange(L, mink, maxk);

printf("删除后的链表: \n");

printL(L);

return 0;

}

3、第三题；

```
#define MAXSIZE 100
```

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
typedef struct{
```

```
int data[MAXSIZE];
```

```
int length;
```

```
}Slist;
```

```
Void inputlist(Slist*a){
```

```
int i=0;
```

```
int n=0;
```

```
a->length=0
```

```
while(scanf("%d",&i)&&i!=-1&&i<MAXSIZE){
```

```
a->data[n]=i;
```

```
a->length++;
```

```
n++;
```

```
}
```

```
}
```

```
void inverslsit(Slist*a){
```

```
int i=0;
```

```
int temp;
```

```
n=data->length;
```

```
while(i n/2){
```

```
temp=a->data[i];
```

```
a->data[i]=a->data[n-i-1];
```

```
a->data[n-i-1]=temp;
```

```
i++;
```

```
}
```

```
}
```

```
int main(){
```

```
Slist*a=(Slist)malloc(sizeof(Slist)); // 要是不想这么办的话，可以直接定义为 Slist 类型  
即可，分配到栈上，传值用 &a
```



```
inputlist(a);  
inverselist(a);  
return 0;  
}
```

4、

```
#include<stdio.h>  
#include<stdlib.h>  
typedef struct Node{  
int data;  
struct Node*next;  
}Node;  
Node* creatL(){  
Node*trail=NULL;  
Node*head=NULL;  
int i=0;  
while(scanf("%d",&i)&&i!=-1){  
Node*newNode=(Node*)malloc(sizeof(Node));  
newNode->next=NULL;  
newNode->data=i;  
if(head==NULL) head=trail=newNode;  
else{  
trail->next=newNode;  
trail=newNode;  
}  
}  
return head;  
}  
Node*merge(Node*a,Node*b){  
Node dummy;  
dummy.next=a;  
Node*p=dummy;
```

```
Node* q=b;
Node* q2=q;
if(a==NULL){
return b;
}
while(q!=NULL&& p->next!=NULL){
if(p->next->data>=q->data){
q2=q->next;
q->next=p->next;
p->next=q;
q=q2;
}
else{
p=p->next;
}
}
if(q==NULL) return dummy.next;
else {
p->next=q;
return dummy.next;
}
void inverse(Node*L){
Node*pre=NULL;
Node*cur=L;
Node*nex=NULL;
while(cur!=NULL){
nex=cur->next;
cur->next=pre;
pre=cur;
cur=nex;
}
```

```

}

int main(){
Node*L1=creatL();
Node*L2=creatl();
L1=merge(L1,l2);
L1=inverse(L1);
return 0;
}

```

4、头插法的做法

```

Node* mergeToDescending(Node* Ha, Node* Hb) {
Node *pa = Ha, *pb = Hb;
Node *pc = NULL; // C表的头指针，利用原节点构建，初始为空
Node *q;    // 用于临时保存下一个节点的指针
while (pa != NULL && pb != NULL) {
if (pa->data <= pb->data) {
// “摘下” pa 节点
q = pa->next; // 1. 记录 pa 的下一个位置，防止断链
pa->next = pc; // 2. 将 pa 的 next 指向 pc（当前已排好的链表头）
pc = pa;    // 3. pc 移动到新的表头
pa = q;    // 4. pa 移向原 A 表的下一个
} else {
// “摘下” pb 节点
q = pb->next;
pb->next = pc; // 同上，头插
pc = pb;
pb = q;
}
}

// 重点：剩余节点也必须逐个头插，才能保证逆序
while (pa != NULL) {
q = pa->next;

```

```

pa->next = pc;

pc = pa;

pa = q;
}

while (pb != NULL) {

q = pb->next;

pb->next = pc;

pc = pb;

pb = q;

}

return pc; // pc 最终指向的就是最大的那个元素，即递减表的头
}

```

5、

```

#include<stdio.h>

#define MAXSIZE 100

typedef struct {

int data[MAXSIZE];

int length;

}Sqlist;

void delete(Sqlist*a,int n){

while(n length-1){

a->data[n]=a->data[n+1];

n++;

}

a->length--;

}

void deletelist(Sqlista,Sqlistb,Sqlist*c){

int n=0;

int i=0;

int j=0;

while(n length&& j length&& j length){

```

```

while(i < length){
    if(b->data[i] < a->data[n]) i++;
    else if(b->data[i] == a->data[n]){
        while(j < length && a->data[n] <= c->data[j]){
            if(c->data[j] < a->data[n]) j++;
            else if(c->data[j] == a->data[n]){
                delete(a,n);
                break;
            }
        }
        else {
            n++;
            break;
        }
    }
    break;
}

else {
    n++;
    break;
}

break;
}

else {
    n++;
    break;
}

}

}

if(i == b->length && j == c->length) break;
}

```

改进方法

```

void deletelist(Sqlist *A, Sqlist *B, Sqlist *C) {
    int i = 0, j = 0, k = 0; // i 扫描 A, j 扫描 B, k 扫描 C
    int newLength = 0; // 用于记录 A 存活元素的下标
    while (i < A->length) {
        // 1. 在 B 中找 A[i]
        while (j < B->length && B->data[j] < A->data[i]) j++;
    }
}

```

```

// 2. 在 C 中找 A[i]
while (k < C->length && C->data[k] < A->data[i]) k++;

// 3. 判断是否满足 “既在 B 又在 C”
// 只有当 B[j] == A[i] 且 C[k] == A[i] 时，才是要删除的元素
if (j < B->length && B->data[j] == A->data[i] &&
    k < C->length && C->data[k] == A->data[i]) {
    // 满足删除条件，跳过该元素（包括 A 中重复的该值）
    int deleteVal = A->data[i];
    while (i < A->length && A->data[i] == deleteVal) {
        i++;
    }
} else {
    // 不满足删除条件，保留该元素
    // 利用 “覆盖法”：将不需要删除的元素移到前面
    A->data[newLength++] = A->data[i++];
}
}

A->length = newLength; // 更新 A 的长度
}

```

6、

```

void ditui(int i){
    if(i==2) printf("%d",i);
    else {
        printf("%d",i);
        ditui(n-1);
    }
}

```

7、

```

#define MAXSIZE 100

#include<stdio.h>

typedef struct {

```

```

int top[2],base[2];

int data[MAXSIZE];

}tws;

void initStack(tws *s) {

s->top[0] = -1;          // 让 top 指向栈顶指针，这样的话需要先移后操作，而且容易判断是否
满空

s->top[1] = MAXSIZE;

}

int push(tws *s, int i, int x) {

// 1. 检查栈满

if (s->top[0] + 1 == s->top[1]) {

return 0; // 栈满，入栈失败

}

// 2. 根据 i 选择栈

if (i == 0) {

s->data[++(s->top[0])] = x; // 栈 0 先加后存

} else if (i == 1) {

s->data[--(s->top[1])] = x; // 栈 1 先减后存

} else {

return 0; // 非法栈号

}

return 1; // 成功

}

int pop(tws *s, int i, int *x) {

if (i == 0) {

if (s->top[0] == -1) return 0; // 栈 0 为空

*x = s->data[(s->top[0])--];

} else if (i == 1) {

if (s->top[1] == MAXSIZE) return 0; // 栈 1 为空

*x = s->data[(s->top[1])++];

} else {

```

```
return 0; // 非法栈号
```

```
}
```

```
return 1; // 成功
```

```
}
```

使用 int *top_1,top_2;

只需要在初始化时:s->top_1=s->data[0];

s->top_2=s->data[MAXSIZE-1];

往后操作只使用指针 top 即可

！！ 若栈顶指针指向当前，则应用 top_1==top_2-1 来判断满

指向后一个，则用 top_1==top_2

8、

```
#include<stdio.h>
```

```
#include<string.h>
```

```
#define MAXSIZE 100
```

```
typedef struct{
```

```
char data[MAXSIZE];
```

```
int top;
```

```
}stack;
```

```
void initstack(stack*s){
```

```
s->top=-1;
```

```
}
```

```
void push(stack*s,char a){
```

```
s->top++;
```

```
s->data[s->top]=a;
```

```
}
```

```
void pop(stack*s,char x){
```

```
*x=s->data[s->top];
```

```
s->top--;
```

```
}
```

```
int compare(char*a){
```

```
int i=0;
```



```

stack s;

initstack(&s);
while(a[i]!='&'){
    push(&s,a[i]);
    i++;
}
i++;
while(1){
    char c;
    if(s.top== -1&&a[i]!='@') return 0;
    if(s.top== -1&&a[i]!='@') break;
    pop(&s,&c);
    if(a[i]!=c) return 0;
    i++;
}
return 1;
}

```

8、完美答案：

```

int compare(char *a) {
    int i = 0;
    stack s;
    initstack(&s);
    // 1. 压栈直到遇到 '&'
    while (a[i] != '&' && a[i] != '\0') {
        push(&s, a[i]);
        i++;
    }
    if (a[i] == '\0') return 0; // 没找到 '&'
    i++; // 跳过 '&'
    // 2. 匹配逻辑
    while (1) {

```

```

// 情况 A: 栈空了
if (s.top == -1) {
    if (a[i] == '@') return 1; // 完美! 栈空且字符串到头
    else return 0;          // 栈空但字符串还没完 (序列 2 长)
}
// 情况 B: 栈没空, 但字符串先到头了 (序列 1 长)
if (a[i] == '@' || a[i] == '\0') return 0;
// 情况 C: 两边都有内容, 进行比对
char c;
pop(&s, &c);
if (a[i] != c) return 0; // 只要有一个不配, 直接枪毙
i++; // 别忘了分号
}
}

```

9、

```

#include<stdio.h>
#define MAXSIZE 100
typedef struct {
    int data[MAXSIZE];
    int rear;
    int front;
    int tag;
}Queue;
int enQueue(Queue*q,int x){
    if(q->tag==1) return 0;
    q->data[q->rear]=x;
    q->rear=(q->rear+1)%MAXSIZE;
    if(q->rear==q->front) q->tag=1;
    return 1;
}
int deQueue(Queue*q,int *x){

```

```

if(q->tag==0) return 0;
*x=q->data[q->front];
q->front=(q->front+1)%MAXSIZE;
if(q->front==q->rear) q->tag=0;
return 1;
}

```

/*2. 两种方法的讨论

通常循环队列有两种处理“判满”的方法：

1. 牺牲 1 个空间：规定 $(rear + 1) \% MAXSIZE == front$ 为满。
2. 设置标志位 (tag)：利用你现在的这种方法。

A. 空间角度

- 不设标志位：会浪费 1 个元素的存储空间。如果 MAXSIZE 很大且元素很小（如 char），这点浪费无所谓。
- 设标志位：利用了全部空间，但多了一个 tag 变量的开销。
- 结论：当队列容量较小且每个元素占空间较多时，设置标志位更好。因为浪费 1 个大元素（比如一个包含很多信息的结构体）的代价，远大于增加一个 int tag 的代价。

B. 时间角度

- 不设标志位：判满只需一次加法和一次取模运算 $(rear + 1) \% MAXSIZE$ 。
- 设标志位：入队 / 出队后多了一次 if 逻辑判断和赋值操作。
- 结论：在现代计算机中，这种时间差异几乎可以忽略不计。但在追求极速的嵌入式底层开发中，不设标志位的逻辑稍微简单一点点。

*/

10、

```
#include<stdio.h>
```

```
#define MAXSIZE 100
```

```
typedef struct{
```

```
int data[MAXSIZE];
```

```
int rear;
```

```
int front;
```

```
}Queue;
```

```
int enQueue(Queue*q,int x){
```

```
if((q->rear+1)%MAXSIZE==q->front) return 0;
```

```
float ave;

if(q->rear==q->front) ave=0.0;

else{

ave=(q->data[q->front]+q->data[(q->rear-1+MAXSIZE)%MAXSIZE])/2.0;

}

if(x<ave){

q->front=(q->front-1+MAXSIZE)%MAXSIZE;

q->data[q->front]=x;

}

else{

q->data[q->rear]=x;

q->rear=(q->rear+1)%MAXSIZE;

}

return 1;

}

int deQueue(Queue*q,int *x){

if(q->front==q->rear) return 0;

*x=q->data[q->front];

q->front=(q->front+1)%MAXSIZE;

return 1;

}
```